



Initiation à la méthodologie DevOps, Gitlab CI/CD et conteneurisation Docker

Partie 2 : Comprendre la CI/CD et construire sa première pipeline

1 - Révisions rapides

Ce qu'on a construit en Jour 1 :

(Voir fichier récap)

- **Docker** : images, conteneurs, Dockerfile. On a dockerisé la Todo API (Node.js/Express + PostgreSQL) et on sait la lancer avec `docker build` / `docker run`.
- **docker-compose** : orchestrer plusieurs conteneurs ensemble (l'API + sa base de données) avec un seul fichier `docker-compose.yml` et les commandes `up` / `down` / `build`.
- **Networks, volumes, variables d'environnement** : les conteneurs communiquent entre eux via un réseau Docker dédié, les données survivent grâce aux volumes, et la configuration (mots de passe, ports) passe par des variables d'environnement plutôt que d'être codée en dur.

On a tout ce qu'il faut côté Docker. Le prochain palier, c'est d'enchaîner ces commandes automatiquement à chaque modification. Avant de coder ça, posons le vocabulaire : CI, CD, DevOps. On en parle tout le temps, mais c'est quoi exactement ?

Planning de la Journée

1. **Échauffement : préparation avant construction** : quatre fichiers cassés, un diagnostic chacun, et la question de l'exposition des ports
 2. **DevOps : d'où ça vient et ce que ça veut dire** : le mur Dev/Ops, les définitions, CALMS, et vingt ans d'histoire en accéléré
 3. **Petite histoire des outils CI/CD** : de Jenkins à GitHub Actions, et ce que chaque génération a changé
 4. **Les grands principes de l'intégration continue** : branches, budget de temps, fail fast, feature flags
 5. **Anatomie d'une pipeline** : les stages, les environnements, les branches Git, manuel vs automatisé
 6. **Trois pipelines réelles, décortiquées** : un site statique, une API avec base de données, un projet multi-services
 7. **DevSecOps** : la sécurité comme un job de plus, et trois attaques qui ont marqué l'industrie
 8. **Vers Kubernetes** : introduction à K3S, la suite du parcours
 9. **À votre tour - Ta première pipeline verte (après-midi)** : ClickFast dockerisé, testé et buildé par GitHub Actions
-

2 - Échauffement : préparation avant construction

Avant de parler de robots qui build à notre place, on va se remettre les mains dedans. Parce qu'automatiser une chaîne qu'on ne sait pas réparer soi-même, c'est le meilleur moyen de se retrouver bloqué devant une pipeline rouge sans savoir quoi regarder.

Ouvrons nos terminaux. Dans ce chapitre, on a 4 fichiers cassés et un diagnostic à poser pour chacun. Je vous invite à écrire vos diagnostics dans le **Journal de bord**, (une section du README du repo d'hier). On débrieфе ensuite tous ensemble.

Journal de bord ? Une section du README où on note, en deux ou trois lignes par entrée, ce qu'on a cherché, ce qu'on a trouvé, et comment on l'a corrigé. C'est la trace écrite du raisonnement, et c'est souvent plus précieux que le code lui-même quand on relit son propre projet six mois plus tard.

L'essentiel du diagnostic

La boîte à outils : cinq commandes qui répondent à cinq questions

Un conteneur qui ne marche pas, ça se diagnostique toujours de la même façon : on pose une question, on tape la commande qui y répond, on avance. Voilà la boîte à outils de base, dont l'essentiel a déjà servi en J1.

```
# Qu'est-ce qui tourne, et depuis quand ?
docker ps -a

# Qu'est-ce que l'application a écrit avant de mourir ?
docker logs mon-conteneur
docker compose logs -f api
docker compose logs --tail=100 --since 30m api

# Comment Docker voit-il ce conteneur (état, réseau, variables) ?
docker inspect mon-conteneur
docker inspect --format='{{.State.Status}}' mon-conteneur
docker inspect --format='{{.NetworkSettings.IPAddress}}' mon-conteneur

# Qu'est-ce que ça consomme, là, maintenant ?
docker stats --no-stream
docker top mon-conteneur

# Et si j'allais voir de l'intérieur ?
docker exec -it mon-conteneur sh
docker run -it --entrypoint sh mon-image # même quand le conteneur crashe au démarrage
```

Deux d'entre elles méritent un mot, parce qu'on ne les a pas croisées en J1.

docker stats ? Un affichage en temps réel de ce que consomment les conteneurs qui tournent : CPU, mémoire, réseau, disque. Avec `--no-stream`, on prend une photo à l'instant t au lieu d'un flux continu. C'est la commande qu'on dégage quand une machine rame sans qu'on sache lequel des huit conteneurs est en cause.

`--entrypoint sh` ? Une façon de démarrer une image en remplaçant la commande de lancement par un shell. Quand un conteneur crashe instantanément, on ne peut pas faire `docker exec` dessus (il n'existe déjà plus). Là, on entre dans l'image telle quelle et on regarde ce qu'il y a dedans : les fichiers sont-ils au bon endroit, le `node_modules` existe-t-il vraiment.

Les erreurs qui reviennent tout le temps

Ces six messages représentent l'écrasante majorité des blocages Docker en début de parcours. Le réflexe à avoir : lire le message *en entier*, le reconnaître, appliquer la contre-mesure.

Erreur	Cause	Solution
<code>port already allocated</code>	Port déjà utilisé sur l'hôte	<code>lsof -i :3000</code> , changer le port hôte
<code>no space left on device</code>	Disque plein	<code>docker system df</code> , puis <code>docker system prune -a</code>
<code>network not found</code>	Network supprimé ou inexistant	<code>docker compose down && docker compose up</code>
<code>service unhealthy</code>	Healthcheck échoue	<code>docker logs</code> + <code>docker inspect</code> pour voir le log du healthcheck
<code>connection refused</code>	Service pas encore prêt	Ajouter <code>depends_on</code> avec <code>service_healthy</code>
<code>unable to find image</code>	Typo ou image inexistante	Vérifier le nom, <code>docker pull</code>

Instant Cheat Sheet! Ce tableau, c'est le premier réflexe de la journée. Une pipeline qui casse cet après-midi affichera exactement les mêmes messages, à un détail près : ils défileront dans les logs d'un runner GitHub au lieu de votre terminal. Le diagnostic, lui, ne change pas.

Quatre fichiers cassés, quatre diagnostics

On y va. Pour chacun, une entrée dans le Journal de bord : *le symptôme observé, la cause, la correction*.

Fichier 1 : l'erreur subtile

```
FROM node:18-alpine
WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD npm start
```

On le build avec `docker build -t debug-1 .` et on regarde ce qui se passe. Deux pistes pour le Journal de bord : ce qui manque dans le `COPY` de la ligne 3 par rapport à ce qu'on a écrit en J1, et pourquoi la forme du `CMD` de la dernière ligne pose problème quand on veut arrêter le conteneur proprement.

Fichier 2 : l'ordre compte

```
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD ["node", "server.js"]
```

Celui-ci fonctionne. Le problème n'est pas un plantage, c'est une lenteur. Le protocole : on build une première fois, on chronomètre, on modifie une ligne dans un fichier source, on rebuild, on chronomètre à nouveau. Ce qu'il faut noter dans le Journal de bord : les deux durées, et l'explication du deuxième chiffre à partir du cache de layers vu en J1.

Fichier 3 : l'image géante

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

On build, puis `docker images debug-3` pour lire la taille. Elle dépasse le gigaoctet. Dans le Journal de bord : au moins trois raisons distinctes à ce poids, et pour chacune, la technique de J1 qui la corrige.

Fichier 4 : le compose qui ne se parle pas

```
services:
  web:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - database
    environment:
      DB_HOST: postgres
      DB_PORT: 5432

  database:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data
```

Celui-là a deux défauts, et ils ne se manifestent pas au même moment. Le premier empêche le fichier d'être lu du tout. Le second laisse la stack démarrer, mais l'application n'atteint jamais sa base de données. Le lancement se fait avec `docker compose -f docker-compose-broken.yml up`, et hop les deux diagnostics vont dans le Journal de bord.

Ptit tips de nouveauté : ce fichier a un troisième défaut, invisible à l'exécution. Beaucoup de `docker-compose.yml` qu'on trouve en ligne commencent par une ligne `version: '3.8'`. Elle ne sert plus à rien depuis des années, et Docker Compose affiche désormais un avertissement quand il la rencontre. Si on la voit dans un fichier, on la retire.

Le cas qui piège tout le monde : EXPOSE contre ports

Ma base de données tourne, mon API la joint sans problème, mais impossible de m'y connecter depuis mon poste avec un client SQL. Pourquoi ?

C'est la confusion la plus fréquente de tout le bloc Docker, et elle vient de deux mots qui ont l'air de dire la même chose.

expose : Documente quel port le conteneur écoute, mais ne l'expose PAS à l'extérieur

ports : Expose réellement le port à la machine hôte

C'est une source de confusion fréquente, alors clarifions.

La directive EXPOSE dans le Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY . .
EXPOSE 3000 # Documentation uniquement !
CMD ["node", "server.js"]
```

Cette ligne EXPOSE 3000 est **purement documentaire**. Elle dit : "Hey, cette application écoute sur le port 3000 à l'intérieur du conteneur"

Mais elle ne rend PAS le port accessible depuis votre machine.

La directive ports dans docker-compose.yml

```
services:
  api:
    build: .
    ports:
      - "3000:3000" # Exposition réelle !
```

Cette ligne crée un **mapping de port** :

- Port 3000 de votre machine → Port 3000 du conteneur
- Maintenant vous pouvez accéder à <http://localhost:3000>

Pourquoi deux mécanismes différents ?

EXPOSE (dans Dockerfile) :

- Documentation pour les développeurs

- Indique l'intention du créateur de l'image
- Utilisé par des outils d'orchestration (Kubernetes)
- N'a aucun effet sur l'accessibilité réseau

ports (dans docker-compose ou -p avec docker run) :

- Exposition réelle du service
- Décision de déploiement (pas de l'image)
- Permet l'accès depuis l'hôte ou l'extérieur

Syntaxes de ports dans docker-compose

Syntaxe courte :

```
ports:
  - "3000:3000" # host:conteneur
  - "8080:80"   # hôte sur 8080, conteneur sur 80
```

Syntaxe longue (plus explicite) :

```
ports:
  - target: 3000      # Port du conteneur
    published: 3000   # Port de l'hôte
    protocol: tcp
```

Exposer sur localhost uniquement (sécurité) :

```
ports:
  - "127.0.0.1:3000:3000" # Accessible seulement depuis la machine locale
```

Port dynamique (assigné par Docker) :

```
ports:
  - "3000" # Docker choisit un port aléatoire sur l'hôte
```

Cas d'usage : quand utiliser expose vs ports

Utilisez expose quand :

- Services internes qui communiquent uniquement entre conteneurs
- Base de données accessible seulement par l'API
- Services dans un même docker-compose

Exemple :

```

services:
  api:
    ports:
      - "3000:3000" # Accessible depuis l'extérieur

  database:
    image: postgres
    expose:
      - "5432" # Seulement accessible par api, pas depuis l'hôte

```

Utilisez ports quand :

- Services qui doivent être accessibles depuis votre machine
- APIs publiques
- Interfaces web
- Services de développement

Sécurité et isolation

Mauvaise pratique :

```

services:
  database:
    image: postgres
    ports:
      - "5432:5432" # La BDD est accessible depuis l'extérieur !

```

N'importe qui sur votre réseau peut tenter de se connecter à votre base de données.

Bonne pratique :

```

services:
  api:
    ports:
      - "3000:3000"

  database:
    image: postgres
    # Pas de ports ! Seulement accessible via le network Docker

```

L'API peut se connecter à la BDD via le network Docker, mais la BDD n'est pas exposée à l'extérieur.

Donc, réponse à la question du début : si le client SQL n'atteint pas la base, c'est très probablement que la base n'a pas de `ports:`. Et c'est une bonne nouvelle, pas un bug. Le jour où on l'expose pour débbugger, on le retire juste après.

Creusons : les ports qu'on n'aurait jamais dû ouvrir

Cette histoire de port de base de données n'est pas un détail de cours. Des dizaines de milliers de bases de données sont accessibles publiquement sur Internet à cet instant précis, souvent sans mot de passe, parce qu'un `ports: - "5432:5432"` a survécu au passage en production. Des moteurs de recherche entiers sont dédiés à les référencer : [Shodan](#) indexe en permanence les services exposés sur Internet, et son rapport annuel liste les bases MongoDB, Redis et PostgreSQL ouvertes par pays.

Shodan ? Un moteur de recherche qui ne cherche pas des pages web, mais des machines connectées : caméras, imprimantes, serveurs de base de données. Il scanne Internet en continu et indexe ce qui répond. C'est fort utile en défense, mais redoutable quand on est du mauvais côté du scan...

La règle qui en découle est simple à énoncer et étonnamment difficile à tenir : **un port publié est une décision de sécurité, pas une commodité de développement**. Trois réflexes qui coûtent une ligne :

- Rien n'est publié par défaut. On ajoute un `ports:` quand on peut nommer *qui*, en dehors de la machine, a besoin d'y accéder.
- Quand on publie pour déboguer, on publie sur `127.0.0.1` uniquement (`"127.0.0.1:5432:5432"`). Le service reste joignable depuis le poste, mais pas depuis le réseau du café d'en face.
- Le port publié en développement et celui de production ne sont pas le même fichier. C'est exactement le rôle des fichiers `docker-compose.override.yml`, qu'on croisera quand on parlera d'environnements.

Cette idée qu'une même application se configure différemment selon l'endroit où elle tourne, on va la retrouver toute la journée : c'est le socle des environnements dev, staging et prod.

On sait réparer ce qu'on a construit à la main. Question suivante, et c'est le vrai sujet de la journée : qui va le construire à notre place, et pourquoi est-ce qu'on a inventé des robots pour ça ? Pour le comprendre, il faut remonter au moment où le métier s'est cassé en deux.

3 - DevOps : d'où ça vient et ce que ça veut dire

Ça vous est déjà arrivé une mise en production qui se passe mal ?

Le souci quand les développeurs et les ops ne se parlent pas;

Imaginez la scène. Le dev a fini sa feature, ça tourne nickel sur son laptop. Il balance un zip à l'équipe ops : "à vous de jouer, mettez ça en prod". L'ops installe, et là... ça casse. Version de Node différente, une variable d'environnement manquante, une dépendance système absente. Chacun accuse l'autre. Le dev dit "ça marche chez moi", l'ops dit "ton code est pourri". Résultat : la prod attend, le client râle.

Ce mur entre Dev et Ops, c'est précisément ce que le DevOps vient casser.

L'essentiel

DevOps, CI, CD : trois mots, trois idées

Posons les définitions une bonne fois.

DevOps : faire collaborer les développeurs (**Dev**) et les opérationnels (**Ops**) pour livrer du code plus vite et plus sûrement. Ce n'est pas un outil, c'est une culture + un ensemble de pratiques. Source de référence : [Atlassian - What is DevOps](#).

CI (Continuous Integration / Intégration Continue) : à chaque modification du code, on l'intègre automatiquement au reste du projet et on lance des vérifications (build + tests). But : détecter tôt les casses/problèmes, pas trois semaines plus tard. Doc : [GitLab - CI/CD](#).

CD (Continuous Delivery/Deployment / Livraison ou Déploiement Continu) : on automatise la suite, jusqu'à la mise en production. En *Delivery*, l'image est prête à déployer et un humain clique sur "go". En *Deployment*, même le "go" est automatique.

La différence Delivery / Deployment est subtile mais elle compte :

Terme	Ce qui est automatique	Le "go" final
Continuous Delivery	build + test + préparation du livrable	un humain valide
Continuous Deployment	tout, jusqu'à la prod	automatique

La plupart des boîtes font de la *Delivery* sur la prod (un humain garde la main pour le déploiement critique) et du *Deployment* complet sur les environnements de test. On y revient dans la section suivante avec les environnements.

Les 3 piliers du DevOps

On l'a vu en J1, mais c'est la colonne vertébrale de la journée, donc on le reprend :

- **Culture** : collaboration entre équipes, responsabilité partagée, feedback continu. Le dev ne "jette" plus son code par-dessus le mur : il est responsable jusqu'en prod.
- **Automation** : tests automatisés, déploiement automatique, infrastructure as code. C'est le cœur de la journée : la pipeline.
- **Measurement** : métriques de performance, monitoring, alerting. On y reviendra plus loin dans le cours, avec Prometheus/Grafana.

Aujourd'hui on attaque surtout le pilier **Automation** : c'est le cœur de la pipeline qu'on construit cet après-midi. Le pilier **Culture**, lui, est invisible dans le code mais omniprésent : la pipeline force tout le monde à travailler proprement (tests verts obligatoires, branches protégées).

CALMS : les 3 piliers, version longue

Trois piliers, c'est ce qu'on retient. Mais quand une entreprise veut *évaluer* où elle en est, elle utilise en général une grille un peu plus fine, appelée **CALMS**. L'acronyme vient de [Jez Humble](#), co-auteur de *Continuous Delivery* et du *DevOps Handbook*, et il ajoute deux lettres à ce qu'on connaît déjà. Référence : [Atlassian - CALMS Framework](#).

- **Culture** : des équipes organisées autour d'un produit, pas autour d'un métier. Pas d'équipe "les devs" d'un côté et "les ops" de l'autre, mais une équipe qui possède son service de bout en bout. La formule qui résume tout : *DevOps résout des problèmes humains, pas des problèmes d'outillage*.
- **Automation** : tout ce qui est répétitif est écrit dans un fichier versionné. Build, tests, déploiement, configuration des serveurs. C'est ce qu'on fait cet après-midi.
- **Lean** : livrer petit et souvent plutôt que gros et rarement, et traiter l'échec comme une information plutôt que comme une faute. Un produit simple aujourd'hui vaut mieux qu'un produit parfait dans six mois.

- **Measurement** : décider sur des chiffres. Combien de temps entre un commit et sa mise en prod, quelle proportion de déploiements casse quelque chose, combien de temps pour se relever. Sans mesure, "on s'améliore" est une opinion.
- **Sharing** : la responsabilité et le savoir circulent. La formule d'Amazon est restée : *you build it, you run it*. Celui qui écrit le code est aussi celui qu'on réveille si ça casse la nuit.

Lean ? Une approche née chez Toyota dans l'industrie automobile : réduire les stocks, les délais et le gaspillage en livrant par petits lots. Transposée au logiciel, ça donne "une petite fonctionnalité livrée tout de suite" plutôt que "vingt fonctionnalités livrées dans six mois".

Les deux lettres ajoutées ne sont pas décoratives. **Lean** explique *pourquoi* on veut une pipeline rapide : si un déploiement coûte cher, on déploie rarement, donc par gros paquets, donc c'est risqué, donc on déploie encore plus rarement. La pipeline casse ce cercle. Et **Sharing** explique pourquoi une pipeline lisible par toute l'équipe vaut mieux qu'un script génial que seule une personne comprend.

Vingt ans d'histoire, en accéléré

Evolution history of DevOps software development

This slide provides glimpse about history of DevOps methodology and how it developed over time from product development practices. It includes phases such as siloed development, agile development, continuous integration & deployment, etc.



This slide is 100% editable. Adapt it to your needs and capture your audience's attention.

Le DevOps n'est pas tombé du ciel en 2009 avec un nom tout fait. C'est une réaction en chaîne, et chaque étape répond à un problème créé par la précédente.

2001 : le Manifeste Agile. Dix-sept personnes se réunissent dans une station de ski de l'Utah et signent le [Manifeste Agile](#). L'idée qui nous intéresse ici : livrer des logiciels qui marchent, fréquemment, par cycles de quelques semaines plutôt que de quelques mois.

Agile ? Une famille de méthodes de gestion de projet (Scrum, Kanban, XP...) qui remplace le gros plan figé sur un an par des cycles courts, avec un livrable fonctionnel à la fin de chacun.

2001-2008 : le mur apparaît. Et c'est là que ça coince. Les devs deviennent effectivement plus rapides : ils livrent toutes les deux semaines. Mais les ops, eux, n'ont pas changé de méthode. Leur métier, c'est la stabilité : chaque changement est un risque, donc on planifie, on gèle, on met en production le samedi soir une fois par trimestre. Deux équipes, deux objectifs qui s'opposent frontalement, séparées par un mur. L'Agile n'a pas créé le mur, mais en accélérant un seul côté, il l'a rendu insupportable.

Juin 2009 : "10+ Deploys per Day". À la conférence Velocity, deux ingénieurs de Flickr, John Allspaw et Paul Hammond, montent sur scène et racontent qu'ils déploient plus de dix fois par jour, en production, sur un site utilisé par des millions de personnes. La première diapo de leur talk est restée célèbre : elle met en scène le dev qui dit "c'est un problème d'ops" et l'ops qui dit "c'est un problème de dev". Leur démonstration : ce n'est pas un problème d'outil, c'est un problème d'organisation, et déployer souvent rend le système *plus* stable, pas moins. Le talk est en ligne : [10+ Deploys per Day, Velocity 2009](#).

Octobre 2009 : le mot existe. Patrick Debois, un consultant belge qui avait vu le talk de Flickr, organise à Gand la première [DevOpsDays](#). Le nom de la conférence devient le nom du mouvement, presque par accident : le hashtag Twitter était `#devops` parce que `#devopsdays` était trop long. On en a parlé en J1, c'est l'acte de naissance officiel du terme.

2013 : The Phoenix Project. Gene Kim, Kevin Behr et George Spafford publient [The Phoenix Project](#), un **roman**. Oui, un roman : on suit un responsable informatique qui hérite d'un projet en train de couler, et le livre lui fait découvrir les principes DevOps chapitre après chapitre. C'est ce qui a fait passer le DevOps du cercle des ingénieurs à celui des directions. Il en tire trois principes, les **Three Ways** :

- **Le flux** : optimiser le mouvement du travail de gauche à droite, du besoin jusqu'au client. Rendre visible ce qui est en cours, réduire la taille des lots, ne jamais faire passer un défaut connu à l'étape suivante.
- **Le feedback** : créer des boucles de retour rapides de droite à gauche. Un problème détecté doit remonter en minutes, pas en semaines. Une pipeline qui échoue en deux minutes, c'est exactement ça.
- **L'expérimentation continue** : instaurer une culture où on ose essayer, où l'échec est une source d'apprentissage, et où on répète les gestes difficiles jusqu'à ce qu'ils deviennent routiniers.

2016 : le SRE de Google. Google publie [Site Reliability Engineering](#), en accès libre. On l'a croisé en J1 : c'est l'approche qui consiste à traiter l'exploitation comme un problème de logiciel, avec des objectifs de fiabilité chiffrés. Sa contribution la plus utile ici, c'est le **budget d'erreur** : si le service vise 99,9 % de disponibilité, on s'autorise explicitement 0,1 % de panne. Tant qu'on est dans le budget, on continue à livrer des nouveautés. Quand on le crève, on arrête tout et on stabilise. Ça remplace la discussion sans fin "on va trop vite / pas assez vite" par un chiffre que tout le monde regarde.

2014 à aujourd'hui : les rapports DORA. Le programme de recherche [DORA](#) interroge chaque année des dizaines de milliers d'équipes. On a vu ses quatre métriques en J1. Ce qui nous intéresse aujourd'hui, c'est ce qu'elles donnent une fois chiffrées, parce que l'écart est vertigineux. Voilà l'ordre de grandeur qui sépare les équipes les plus performantes des moins performantes, d'après l'[Accelerate State of DevOps Report](#) :



Métrique	Équipes "elite"	Équipes "low"
Fréquence de déploiement	à la demande, plusieurs fois par jour	une fois par mois à une fois par semaine

Métrique	Équipes "elite"	Équipes "low"
Lead time (commit vers prod)	moins d'un jour	de un à six mois
Taux d'échec des changements	autour de 5 %	plusieurs dizaines de %
Temps de rétablissement	moins d'une heure	de une semaine à un mois

Les seuils exacts bougent d'une édition à l'autre du rapport, et le classement est plus nuancé qu'un simple bon/mauvais. Mais l'ordre de grandeur, lui, ne bouge pas : on parle d'un facteur cent, voire mille, sur le temps entre un commit et sa mise en production. Pas de vingt pour cent.

Lead time ? Le temps écoulé entre le moment où on valide un changement (le commit) et le moment où les utilisateurs en bénéficient réellement. C'est la métrique qui mesure la longueur de toute la chaîne, pas la vitesse d'un maillon.

Et une nouveauté, parce que le rapport continue d'évoluer : DORA suit désormais une **cinquième** métrique, le *deployment rework rate*, c'est-à-dire la proportion de déploiements qui obligent à intervenir en catastrophe juste après (rollback, correctif à chaud). Autrement dit : livrer souvent, oui, mais pas en repassant derrière soi à chaque fois.

Pour aller plus loin : ce que le DevOps a fait au métier

Ce mouvement a redessiné des fiches de poste entières, et ça vaut le coup de savoir ce que recouvrent les intitulés qu'on va croiser en cherchant un stage ou une alternance.

- **DevOps Engineer** : le titre le plus répandu, et le plus flou. Dans les faits, c'est souvent la personne qui construit et maintient la chaîne de livraison : pipelines, images, environnements, outillage des équipes de dev.
- **SRE (Site Reliability Engineer)** : orienté fiabilité du service en production. Objectifs chiffrés, astreinte, post-mortems. Historiquement un poste Google, aujourd'hui répandu dans toutes les grandes plateformes.
- **Platform Engineer** : le titre qui monte le plus vite. L'idée est de construire une plateforme interne que les équipes de dev consomment en libre-service, au lieu de venir demander à chaque fois. On parle de *plateforme développeur interne*.
- **Cloud Engineer / Infrastructure Engineer** : centré sur l'infrastructure elle-même, souvent chez un fournisseur cloud, avec de l'Infrastructure as Code (Terraform, Ansible).

Un point qui surprend souvent : ces postes sont rarement des postes juniors purs. Ils supposent d'avoir déjà développé, déjà cassé des choses en production, déjà été de l'autre côté. Ce que ça veut dire concrètement pour la suite du parcours : la meilleure porte d'entrée, c'est un poste de dev où on prend en charge la pipeline de son équipe. Ce qu'on fait cet après-midi, sur un petit projet, c'est exactement ce geste-là en miniature.

Bonus perso : dans les conversations d'entreprise, on entend beaucoup "on fait du DevOps" pour dire "on a un Jenkins". 🙌 Petit défi hors-cours : à la prochaine occasion (entretien, stage, discussion avec un pro), demander : "vous déployez à quelle fréquence, et qui appuie sur le bouton ?" La réponse en dit plus long que n'importe quelle liste d'outils.

On a le vocabulaire et l'histoire des idées. Voyons l'histoire des outils, et elle raconte quelque chose de différent : comment on est passé d'un serveur qu'on bichonne à un fichier texte dans le repo.

4 - Petite histoire des outils CI/CD

Quelqu'un a déjà entendu parler de Jenkins ? De Travis ?



(pas Travis Scott non)

Pour comprendre pourquoi GitLab-CI et GitHub Actions existent, faut savoir d'où on vient. C'est une histoire de "on automatise de plus en plus, et c'est de plus en plus simple".

L'essentiel

- [CruiseControl](#) (2001, ThoughtWorks) : l'ancêtre de l'ancêtre. Un outil Java qui surveille le dépôt de code, et qui relance une compilation complète dès qu'il détecte un changement. Configuration en XML, à la main. C'est la première fois qu'on automatise l'idée "à chaque changement, on vérifie que tout compile encore".
- [Hudson](#) (2005), devenu [Jenkins](#) (2011) : l'ancêtre, le grand-père de la CI. Un serveur que vous installez et maintenez vous-même, avec des centaines de plugins. Surpuissant, mais lourd : il faut une machine dédiée, gérer les mises à jour, configurer chaque job à la main. On en trouve encore partout en entreprise. Le passage de Hudson à Jenkins, en 2011, est une histoire de gouvernance : après le rachat de Sun par Oracle, la communauté s'est brouillée avec le nouveau propriétaire du nom, a forké le projet et l'a rebaptisé.
- [Travis CI](#) (2011) : la révolution du "CI as a Service". Plus besoin d'installer un serveur : vous mettez un fichier `.travis.yml` dans votre repo, et Travis exécute la pipeline sur ses machines. Énorme dans l'écosystème open-source des années 2010.
- [CircleCI](#) (2011) : même génération, même promesse, avec une insistance particulière sur la vitesse et la parallélisation des jobs. C'est aussi l'un des premiers à populariser l'exécution de chaque job dans un conteneur Docker isolé.
- [GitLab CI/CD](#) (2015) : GitLab intègre la CI directement dans la plateforme Git. Votre code, vos merge requests, vos pipelines : tout au même endroit. Un seul fichier `.gitlab-ci.yml` à la racine, et c'est parti.
- [GitHub Actions](#) (2019) : la réponse de GitHub. Même idée que GitLab-CI, intégrée à GitHub, avec une marketplace énorme d'actions réutilisables. Des fichiers `.yml` dans `.github/workflows/`.

La tendance est claire : **de "j'installe et je maintiens un serveur" (Jenkins) vers "je mets un fichier YAML dans mon repo et la plateforme s'occupe du reste" (GitLab/GitHub)**. C'est pour ça qu'aujourd'hui on travaille avec GitHub Actions et, plus tard dans le cours, si on souhaite GitLab-CI. Nous ne ferons pas de Jenkins mais vous pourrez avoir des ressources. Allez, moins de plomberie, plus de valeur.

Ce que chaque génération a réellement changé

Retenir les dates ne sert pas à grand-chose. Ce qui compte, c'est le problème que chaque génération a réglé, parce que le problème d'aujourd'hui est l'héritage direct du dernier de la liste.

Génération	La promesse	Ce qui devient possible	Ce qui reste pénible
CruiseControl, Hudson, Jenkins	"un serveur surveille le dépôt et lance le build"	l'intégration ne dépend plus d'un humain qui y pense	il faut installer, patcher et sauvegarder ce serveur, et sa configuration vit dans son interface web
Travis, CircleCI	"un service en ligne exécute ma pipeline"	plus de serveur à maintenir, et la config passe dans un fichier du repo	le service est extérieur au dépôt, avec des allers-retours entre deux outils
GitLab-CI, GitHub Actions	"ma pipeline vit dans mon repo, avec mon code"	la config est versionnée, revue en merge request, et suit les branches	le YAML devient un langage à part entière, avec ses pièges
Runners conteneurisés	"chaque job tourne dans son propre conteneur"	environnement d'exécution reproductible et jetable	il faut comprendre Docker pour comprendre sa CI

La dernière ligne est celle qui nous concerne le plus directement. **La CI moderne suppose Docker**. Chaque job de la pipeline qu'on écrit cet après-midi démarre dans un conteneur neuf, exactement comme ceux qu'on lance depuis hier. C'est pour ça que le bloc Docker venait en premier dans ce cours : sans lui, la CI ressemble à de la magie noire.

Un détail qui a son importance : la configuration versionnée change la nature du travail. Modifier une pipeline devient une modification de code, donc ça passe par une branche, une merge request, une relecture, et ça s'annule avec un `git revert`. Sur un Jenkins configuré à la souris, personne ne sait qui a changé quoi ni pourquoi.

Approfondissons : et Jenkins, alors, c'est mort ?

Non, et c'est important de le savoir avant un entretien. Jenkins reste massivement présent en entreprise, particulièrement dans la banque, l'assurance, l'industrie et le secteur public. Trois raisons, dont aucune n'est technique :

- **L'inertie** : une entreprise avec quatre cents pipelines Jenkins ne migre pas en un trimestre. Le coût du changement dépasse largement le gain.
- **L'auto-hébergement** : certaines organisations ne peuvent pas envoyer leur code chez GitHub ou GitLab.com, pour des raisons réglementaires. Jenkins tourne sur leurs machines, dans leur réseau, point.

- **L'extensibilité extrême** : quand il faut piloter un automate industriel ou un mainframe depuis la CI, on trouve un plugin Jenkins et on ne trouve rien d'autre.

À noter tout de même : Jenkins a rattrapé une partie du retard avec le **Jenkinsfile**, un fichier de configuration placé dans le repo, qui reprend l'idée de la configuration versionnée. La différence de fond avec GitHub Actions ou GitLab-CI se réduit donc, même si le serveur reste à maintenir.

Ce qu'il faut en retenir pour la suite : la syntaxe change d'un outil à l'autre, les concepts ne changent pas. Un stage, un job, un runner, un artefact, un cache, une variable secrète. Quand on aura fait GitHub Actions cet après-midi puis GitLab-CI, passer à un troisième outil sera une affaire de documentation, pas d'apprentissage.

On sait d'où viennent les outils. Mais un outil ne fait pas une pratique : on peut très bien avoir une pipeline et ne pas faire d'intégration continue du tout. Voyons donc ce que "faire de la CI" veut dire concrètement, au quotidien, dans une équipe.

5 - Les grands principes de l'intégration continue

Une équipe a un serveur de CI, des pipelines qui tournent, des tests automatisés. Est-ce qu'elle fait de l'intégration continue ?

Pas forcément. Et ce n'est pas une question piège : c'est le malentendu le plus répandu du domaine. On peut avoir tout l'outillage du monde et travailler exactement comme avant. Si chaque développeur code trois semaines sur sa branche avant de la fusionner, il n'y a **aucune intégration continue**, même avec la plus belle pipeline du marché. La pipeline vérifie des branches isolées les unes des autres, et le vrai moment de vérité (est-ce que tout ce travail tient ensemble ?) arrive toujours aussi tard, et toujours aussi douloureusement.

L'intégration continue est une **pratique d'équipe**, l'outil ne fait que la rendre supportable. La référence sur le sujet reste l'article de [Martin Fowler sur l'intégration continue](#), régulièrement mis à jour depuis 2000. Voilà ce qu'il faut en garder.

L'essentiel des principes

Une branche de référence, et elle reste déployable

Tout part de là. Il existe **une** branche qui représente l'état du produit, en général `main`. La règle qui la protège tient en une phrase : *à n'importe quel instant, on doit pouvoir prendre `main` et la mettre en production*.

Ce n'est pas un idéal théorique, c'est une contrainte opérationnelle. Le jour où une faille de sécurité critique tombe et qu'il faut livrer un correctif en deux heures, on n'a pas le temps de démêler ce qui est fini de ce qui ne l'est pas. Soit `main` est déployable, soit on passe la nuit dessus.

Ce qui en découle, et qui surprend souvent : **une pipeline rouge sur `main` n'est pas un problème personnel, c'est un arrêt d'usine**. Tant que la branche de référence est cassée, personne dans l'équipe ne peut savoir si son propre travail marche. Fowler le formule sans détour : réparer un build cassé devient la priorité numéro un de l'équipe, avant toute nouvelle fonctionnalité.

Intégrer souvent, en petits morceaux

Deuxième principe, et c'est celui qui fait le plus mal à changer : chacun pousse son travail sur la branche de référence **au moins une fois par jour**.

Mais attends, du coup, comment je fais si ma feature est pas finie ? Bonne question, et on y répond deux paragraphes plus bas avec les feature flags. Gardons d'abord le pourquoi.

Comparons deux façons de travailler sur la même fonctionnalité :

	Une branche de trois semaines	Des intégrations quotidiennes
Conflits Git à la fusion	des dizaines, sur des fichiers modifiés par tout le monde	quelques-uns, sur ce qui a bougé la veille
Quand on découvre l'incompatibilité	à la fin, quand tout est écrit	le lendemain de la divergence
Coût d'un retour arrière	trois semaines de travail à démêler	une journée
Ce que la pipeline vérifie	une branche qui ne ressemble plus à la prod	l'état réel du produit

Le coût d'une fusion ne grandit pas de façon linéaire avec le temps. Deux branches qui divergent d'un jour, ça se recolle en cinq minutes. Deux branches qui divergent de trois semaines, ça se recolle en une journée, avec des décisions arbitraires ("je garde ma version ou la sienne ?") prises sous pression et sans contexte. C'est ce qu'on appelle le **merge hell**, et tout le monde l'a vécu au moins une fois.

Merge hell ? Le moment où fusionner deux branches devient plus long et plus risqué que d'avoir écrit le code. Symptôme classique : on résout un conflit, ça en fait apparaître trois autres, et on n'est plus sûr de ce que fait le code final.

Trunk-based ou GitFlow : le vrai débat

Deux modèles de branches s'opposent, et le choix a des conséquences directes sur la pipeline.

GitFlow, publié par Vincent Driessen en 2010, propose un jeu de branches à long terme : `main`, `develop`, des branches `feature/*`, `release/*`, `hotfix/*`. C'est structuré, ça rassure, et c'est devenu le modèle par défaut d'une génération entière de développeurs.

Trunk-based development dit l'inverse : une seule branche à long terme, et des branches de fonctionnalité qui vivent moins de deux jours avant d'être fusionnées.

Trunk ? Le tronc, c'est-à-dire la branche principale. "Trunk-based" veut dire "tout le monde travaille sur le tronc, ou tout près".

Ce qui est intéressant, c'est que l'auteur de GitFlow lui-même a ajouté un avertissement en tête de son article : le modèle a été conçu pour du logiciel **versionné et distribué** (une application qu'on télécharge, avec des versions 1.2 et 1.3 maintenues en parallèle). Pour une application web déployée en continu, il le déconseille explicitement.

Le point de bascule est là : *est-ce que je dois maintenir plusieurs versions en même temps ?* Un logiciel embarqué ou une bibliothèque open source, oui. Une API web, non : il n'y a qu'une version en ligne, celle de maintenant.

Et il y a une raison mécanique de préférer le trunk-based en CI. Avec GitFlow, la pipeline valide `develop`, qui n'est pas ce qui part en production. Avec le trunk-based, la pipeline valide exactement ce qui va être déployé. La question "est-ce que le vert de la CI garantit quelque chose ?" n'a une bonne réponse que dans le second cas.

Un artefact construit une fois, promu d'environnement en environnement

Voilà le principe le plus souvent violé, et celui qui produit les pannes les plus incompréhensibles.

Ce qu'il ne faut pas faire :

```
# On build en staging
docker build -t todo-api:staging .

# ... tests OK ...

# Puis on rebuild pour la prod
docker build -t todo-api:prod .
```

Deux builds, donc potentiellement **deux images différentes**. Entre les deux, une dépendance npm a pu publier un correctif, une image de base a pu être mise à jour, le cache a pu se comporter autrement. Ce qui a été testé en staging n'est plus tout à fait ce qui tourne en prod, et le jour où ça casse, on cherche pendant des heures un bug qui n'existe que dans un des deux artefacts.

Ce qu'il faut faire :

```
# Un seul build, identifié par le commit exact
docker build -t todo-api:a1b2c3d .
docker push todo-api:a1b2c3d

# Ensuite, on promeut la MÊME image
# staging : on déploie a1b2c3d
# prod    : on déploie a1b2c3d, exactement le même sha
```

Artefact ? Le livrable produit par le build : ici une image Docker, ailleurs un `.jar`, un binaire, une archive. C'est l'objet qu'on déplace d'un environnement à l'autre.

Promouvoir ? Faire passer un artefact déjà construit à l'environnement suivant, sans le reconstruire. On promeut un artefact comme on promeut quelqu'un : c'est le même, il change juste de place.

Le tag basé sur le hash du commit n'est pas un détail cosmétique. Il rend la question "quelle version tourne en prod ?" vérifiable en une commande, et rejouable : on retrouve le commit exact, on relit le code exact.

Le budget de temps de la pipeline

Fowler donne un chiffre qui a l'air arbitraire et qui ne l'est pas : **une pipeline doit rendre son verdict en dix minutes environ**.

Le raisonnement est comportemental, pas technique. Au-delà de dix minutes, on ne reste pas à attendre : on part sur autre chose. Et quand le résultat arrive, on a perdu le contexte, on doit revenir en arrière, retrouver ce qu'on était en train de faire. Le coût réel du retard n'est pas les minutes d'attente, c'est le changement de contexte. Pire, une pipeline lente pousse à intégrer moins souvent, ce qui détruit le principe précédent.

Comment on tient ce budget ? Avec la **pyramide des tests**, décrite dans [The Practical Test Pyramid](#) de Ham Vocke.

- **Tests unitaires** : ils vérifient une fonction isolée, sans base de données ni réseau. Quelques millisecondes chacun. On en a beaucoup, ils tournent à chaque commit.
- **Tests d'intégration** : ils vérifient que deux briques se parlent correctement, par exemple l'API et sa base de données. Quelques secondes chacun. On en a nettement moins.

- **Tests de bout en bout** : ils pilotent l'application complète comme le ferait un utilisateur, navigateur compris. Plusieurs dizaines de secondes chacun, et ils cassent pour de mauvaises raisons (un bouton déplacé de trois pixels). On en garde une poignée, sur les parcours critiques uniquement.

La forme de pyramide dit la proportion : beaucoup en bas, très peu en haut. La forme inverse, beaucoup de tests de bout en bout et peu d'unitaires, porte un nom dans le métier : le **cornet de glace**, et c'est un cauchemar à maintenir.

Voilà un budget réaliste pour une pipeline d'API :

Stage	Durée cible	Ce qui la fait déraper
lint	10 à 30 s	rien, c'est le stage le moins cher
tests unitaires	1 à 2 min	des tests qui touchent la base ou le réseau sans le dire
build de l'image	1 à 3 min	pas de cache de layers, <code>COPY . .</code> en première ligne
tests d'intégration	2 à 4 min	attendre bêtement que la base démarre, sans healthcheck
push	30 s à 1 min	une image de 1,2 Go au lieu de 150 Mo

On retrouve exactement les optimisations de J1 : le cache de layers et la taille de l'image ne sont plus une question d'esthétique, ils sont dans le budget de temps de l'équipe entière, à chaque push de chaque personne, toute la journée.

Fail fast : échouer tôt et pour pas cher

Corollaire direct du budget : on ordonne les étapes de la moins chère à la plus chère. Si le linter casse en quinze secondes, inutile d'avoir dépensé trois minutes à build une image. On y revient en détail dans le chapitre suivant, quand on regardera l'anatomie d'une pipeline.

Découpler le déploiement de la mise à disposition

Revenons à la question laissée en suspens : comment intégrer tous les jours une fonctionnalité qui demande trois semaines de travail ?

La réponse tient en une distinction que beaucoup d'équipes n'ont jamais faite explicitement.

- **Déployer** : le code est en production, il tourne sur le serveur.
- **Mettre à disposition (release)** : les utilisateurs peuvent s'en servir.

Ces deux choses n'ont aucune raison d'arriver en même temps. On peut très bien déployer du code que personne ne voit encore. C'est le rôle des **feature flags**, décrits par Pete Hodgson dans [Feature Toggles](#).

Feature flag (ou feature toggle) ? Un interrupteur, souvent une simple variable d'environnement, qui active ou désactive une fonctionnalité sans redéployer. Le code de la fonctionnalité est en production, mais il ne s'exécute que si l'interrupteur est sur "on".

Concrètement, dans du code Node :

```
// La nouvelle recherche est déployée mais dormante :  
// on l'allume pour l'équipe d'abord, pour tout le monde ensuite.  
if (process.env.FEATURE_NEW_SEARCH === 'true') {  
  return newSearch(query);  
}  
return legacySearch(query);
```

Ce que ça change dans la vie de l'équipe :

- On intègre tous les jours, même une fonctionnalité à moitié écrite, puisqu'elle est éteinte.
- On allume progressivement : l'équipe interne, puis 5 % des utilisateurs, puis tout le monde.
- Quand ça se passe mal, on éteint l'interrupteur en quelques secondes, sans redéployer, sans rollback.

Le prix à payer, et il est réel : chaque flag est une branche de plus dans le code. Un flag qu'on oublie de retirer après six mois devient une bombe à retardement, parce que plus personne ne sait ce que fait la moitié éteinte. La discipline qui va avec : un flag a une date de péremption, et on le supprime dès que la fonctionnalité est définitivement en place.

Pour les curieux : combien coûte un bug selon le moment où on le trouve

Pourquoi est-ce qu'on insiste tellement pour détecter tôt ? Un bug est un bug, non ?

Non, et c'est mesurable. Le même défaut, exactement le même, coûte des ordres de grandeur différents selon l'endroit de la chaîne où on le rencontre. Regardons ce que ça donne, en temps humain, sur une équipe réelle :

Où le bug est trouvé	Ce qu'il faut faire	Coût en temps humain
Dans l'éditeur (le linter souligne)	corriger la ligne	quelques secondes
En CI, sur la branche de la personne	relire les logs, corriger, re-push	quelques minutes
En revue de code par un collègue	expliquer, discuter, corriger, refaire une revue	une demi-heure, à deux
En staging, après fusion	reproduire, bisecter parmi plusieurs commits, corriger	une demi-journée
En production, signalé par un utilisateur	reproduire sans les données, correctif à chaud, communication, post-mortem	plusieurs jours, plusieurs personnes

Deux effets se combinent. D'abord le **contexte** : quand on trouve le bug dans la minute, on a encore tout le raisonnement en tête. Trois semaines plus tard, on relit son propre code comme celui d'un inconnu. Ensuite le **nombre de personnes** : un bug en production mobilise le support, l'astreinte, parfois la communication. Un bug attrapé par le linter ne mobilise personne.

Il y a un troisième effet, moins visible et plus sournois : ce qu'un bug en production coûte en **confiance**. Une équipe qui casse la prod régulièrement finit par déployer moins souvent pour se protéger. Elle déploie donc par plus gros paquets, donc plus risqués, donc elle casse plus. C'est exactement le cercle vicieux que les métriques DORA mesurent, et la pipeline est ce qui permet d'en sortir.

Voilà pourquoi une pipeline n'est pas juste un confort d'outillage : c'est le mécanisme qui déplace la détection des bugs vers la gauche du tableau, là où ils sont bon marché.

On a les principes. Passons au concret : à quoi ressemble une pipeline, étape par étape ? C'est ce qu'on décortique tout de suite, parce qu'on va devoir choisir nous-mêmes l'ordre des étapes dans nos fichiers YAML.

6 - Anatomie d'une pipeline

Mais attends, du coup, j'ai pas compris : c'est quoi une pipeline ?

Une **pipeline**, c'est une suite d'étapes automatiques déclenchées par un événement (en général : un `git push`). Chaque étape fait une chose, et si une étape échoue, la pipeline s'arrête : on ne déploie jamais du code cassé.

Pipeline ? Une chaîne de montage pour votre code. À l'entrée : votre commit. À la sortie : votre app en prod. Entre les deux, des stations de contrôle automatiques (tests, build...) qui bloquent tout si quelque chose cloche.

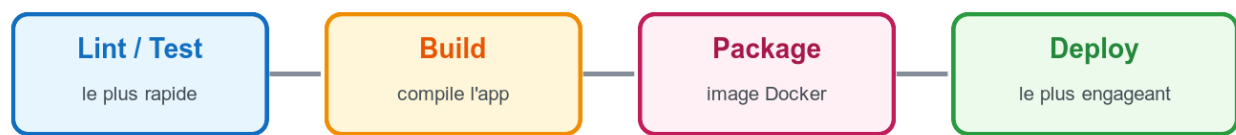
Concrètement, ce que ça vous apporte :

- **On attrape les bugs tôt** : les tests tournent à chaque push. Un test cassé, et vous le savez en 2 minutes, pas en prod.
- **Fini "ça marche chez moi"** : la pipeline build et teste dans un environnement neutre, identique pour tout le monde.
- **Déploiements reproductibles** : la même suite d'étapes, à chaque fois. Pas d'oubli, pas d'étape sautée à 23h un vendredi.
- **Tout le monde peut déployer** : plus besoin du "gourou" qui seul sait comment mettre en prod. Un push suffit.
- **Traçabilité** : chaque pipeline garde un log. Qui a déployé quoi, quand, et est-ce que ça a marché.

Bonus perso : cette pipeline qu'on va brancher aujourd'hui, Netflix en déclenche plusieurs milliers par jour. Le principe est exactement le même que le vôtre, juste à une autre échelle. 🙌 Mini-challenge hors-cours : trouvez une offre d'emploi DevOps d'une boîte qui vous attire, et listez les outils CI/CD qu'ils demandent. Vous verrez GitLab-CI, GitHub Actions, ou Jenkins quasi à chaque fois.

Regardons maintenant ce qu'il y a *dedans*. Une pipeline, c'est une séquence de **stages** (étapes), et chaque stage contient un ou plusieurs **jobs** (tâches).

Anatomie d'une pipeline : stages & jobs



un stage = plusieurs jobs



Si un stage échoue, les suivants ne tournent pas.

L'essentiel

Les stages classiques d'une pipeline

L'ordre compte : on ne build pas si les tests cassent, on ne déploie pas si le build casse. Voilà la séquence typique, du plus rapide au plus engageant :

1. **lint** : on vérifie le style du code (formatage, conventions). Ultra rapide, ça casse en premier si le code est mal foutu.
2. **test** : on lance les tests unitaires et d'intégration. Si un test échoue, stop.
3. **build** : on construit l'image Docker. Si le build échoue (Dockerfile cassé), stop.
4. **push** : on pousse l'image sur le registry (DockerHub). Maintenant elle est disponible partout.
5. **deploy** : on déploie l'image sur le serveur / le cluster Kubernetes.

Lint ? Un outil (ex : [ESLint](#) pour JavaScript) qui analyse le code sans l'exécuter, pour repérer les erreurs de style et les bugs évidents (variable jamais utilisée, point-virgule oublié). "Linter", c'est passer le code au peigne fin.

Pourquoi cet ordre, et pas un autre ? **On met le moins coûteux et le plus susceptible de casser en premier.** Linter prend 5 secondes. Si le code est mal formaté, inutile de gaspiller 3 minutes à builder une image Docker. On échoue vite, on échoue pas cher. C'est le principe du **fail fast**.

Les environnements : dev, staging, prod

Avant de coder une pipeline, faut comprendre où on déploie. Quand on bosse sur un vrai projet avec plusieurs "embouts" accessibles en ligne, on a en général trois environnements :

- **dev (développement)** : votre machine, ou un environnement de test rapide. Ça casse souvent, c'est normal, c'est fait pour ça.
- **staging (pré-production)** : une copie aussi fidèle que possible de la prod. On y teste les nouvelles features une fois fusionnées, avant de les pousser pour de vrai. C'est le filet de sécurité.
- **prod (production)** : le vrai serveur, celui que vos utilisateurs touchent. Tout ce qui arrive ici doit être propre et testé.

Staging (ou préprod) ? L'environnement miroir de la prod. Même base de données (en copie), même config, mais sans vrais utilisateurs. Si ça pète en staging, personne ne le voit. Si ça pète en prod, tout le monde le voit (et ça fait mal).

L'idée : votre pipeline déploie automatiquement en staging à chaque merge, mais le déploiement en prod demande souvent une validation manuelle (Continuous Delivery, on l'a vu dans la section précédente).

Les branches Git : les bonnes pratiques

La pipeline est intimement liée à votre stratégie de branches Git. Petit récap, parce que c'est ce qui décide quand et où on déploie :

- **master / main** : la branche à garder **protégée**. Tout ce qui arrive ici doit être totalement propre et fonctionnel. C'est elle qui déclenche le déploiement.
- **staging** (OU `pre-production`, OU `development`) : idéalement on a une branche pour rassembler le travail des développeurs de l'équipe. On vérifie que toutes les nouvelles fonctionnalités fonctionnent une fois fusionnées.
- **autres branches** (`feature/xxx`, `fix/yyy`) : c'est super important de bosser sur des branches à part, pour ne pas polluer ce qui marche déjà. Une fois le travail satisfaisant, on push la branche et on ouvre une **merge request** (ou pull request sur GitHub). Elle sera *reviewed* (revue) par un autre dev avant fusion. Une vérification humaine en plus du robot.

Merge request (MR) ? Sur GitLab, une demande de fusion d'une branche vers une autre. C'est le moment où un collègue relit votre code avant qu'il rejoigne `main`. Sur GitHub on dit "pull request" (PR). Même chose.

On exploitera ça concrètement plus tard dans le cours, quand on configurera une pipeline pour **ne déployer que depuis main**. Une feature en cours sur une branche `feature/xxx` lance les tests, mais ne déploie rien. Logique : on ne met pas du travail en chantier en prod.

Avant / Après : le déploiement manuel vs la pipeline

Pour bien sentir le gain, comparons ce que vous avez fait à la main en J1 avec ce qu'on va automatiser.

Avant (déploiement manuel) :

```
# À chaque modif de la Todo API, vous tapiez à la main :
npm test                # je teste (si j'y pense)
docker build -t monpseudo/todo-api:1.0 . # je build
docker push monpseudo/todo-api:1.0      # je pousse sur DockerHub
ssh user@serveur        # je me connecte au serveur
docker pull monpseudo/todo-api:1.0      # je récupère l'image
docker stop todo-api && docker rm todo-api
docker run -d -p 3000:3000 monpseudo/todo-api:1.0 # je relance
```

Sept commandes, dans le bon ordre, sans en oublier une, à chaque déploiement. Et si vous oubliez le `npm test` ? Vous déployez peut-être du code cassé.

Après (pipeline) :

```
git push origin main
```

C'est tout. Le robot fait les sept étapes, dans le bon ordre, à tous les coups, et il refuse de déployer si les tests cassent.

Verdict : on échange sept commandes fragiles contre un `git push` infallible. C'est ça, la CI/CD.

Creusons : que se passe-t-il quand un stage échoue ?

Un point qui surprend souvent les débutants : *que devient ma pipeline si un test casse ?*

La règle d'or : **un stage qui échoue arrête la pipeline**. Les stages suivants ne tournent pas du tout. Si le stage `test` est rouge, le stage `build` n'est jamais lancé. C'est voulu : on ne veut surtout pas builder et déployer une image dont on sait qu'elle est cassée.

Quelques cas concrets de "ce qui casse et pourquoi" :

- **Un test unitaire échoue** → stage `test` rouge → pas de build, pas de déploiement. Vous corrigez, vous re-push, la pipeline repart de zéro.
- **Le Dockerfile a une erreur** (typo dans `FROM`, fichier manquant) → stage `build` rouge → pas de push. L'image cassée ne part jamais sur DockerHub.
- **Les identifiants DockerHub sont faux** → stage `push` rouge → l'image est buildée mais reste sur le runner. Vous corrigerez vos secrets plus tard dans le cours.
- **Le serveur de déploiement est injoignable** → stage `deploy` rouge → l'app reste sur l'ancienne version. La prod ne tombe pas, elle reste juste sur le code précédent.

Ce dernier point est crucial : une pipeline bien faite **ne casse jamais la prod**. Au pire, elle refuse de déployer la nouvelle version, et l'ancienne continue de tourner. On y reviendra avec le **rolling update** de Kubernetes, plus tard dans le cours, qui pousse cette idée encore plus loin.

La théorie tient en une image : des stages ordonnés qui s'arrêtent au premier échec. Mais une pipeline réelle, dans un vrai repo, ne ressemble jamais tout à fait au schéma. On va donc en ouvrir trois, de la plus simple à la plus tordue, et regarder ce qu'il y a écrit dedans.

7 - Trois pipelines réelles, décortiquées

Ces trois pipelines sont écrites en **GitHub Actions**, l'outil qu'on utilise cet après-midi. Aucune ne correspond au projet de l'après-midi : ce sont trois autres projets, choisis parce qu'ils montrent chacun une marche de plus. L'idée, c'est d'avoir vu la syntaxe et compris les mécanismes avant d'écrire les siens.

Workflow ? Le nom que GitHub Actions donne à une pipeline. C'est un fichier YAML rangé dans `.github/workflows/`. Un repo peut en contenir plusieurs, chacun avec son propre déclencheur. Doc : [GitHub Actions](#).

Runner ? La machine (virtuelle) qui exécute vos jobs. `ubuntu-latest`, c'est un runner Ubuntu fourni gratuitement par GitHub. Sur GitLab, ce sont les "shared runners". Point important : c'est une machine **neuve à chaque exécution**, qui ne connaît ni votre code ni vos outils tant que vous ne les y avez pas mis.

L'essentiel des trois cas

Cas 1 : un site de documentation statique, deux jobs, 40 secondes

Le contexte. Une équipe maintient la documentation d'un produit avec [Eleventy](#), un générateur de site statique : on écrit des fichiers Markdown, l'outil produit un dossier de HTML. Le site est publié sur [GitHub Pages](#). Zéro base de données, zéro secret, zéro conteneur.

```

name: Docs

# Deux déclencheurs différents, et c'est là que se joue la logique :
# une PR se contente de vérifier, un push sur main publie.
on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      # Le runner est vide : sans checkout, il n'a même pas notre code.
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      with:
        node-version: '20'
        cache: 'npm'
      - run: npm ci
      - run: npm run build          # produit le dossier _site/
      # On range le résultat pour que le job suivant puisse le reprendre.
      - uses: actions/upload-artifact@v4
      with:
        name: site
        path: _site/

  deploy:
    needs: build                      # ne démarre que si build est vert
    if: github.ref == 'refs/heads/main' # et seulement sur main
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
      with:
        name: site
      - uses: actions/deploy-pages@v4

```

Ce qu'on lit dedans. Trois mécanismes, et on les retrouvera partout :

- `uses:` appelle une **action** réutilisable, c'est-à-dire un bout de pipeline écrit par quelqu'un d'autre et partagé. `actions/checkout@v4` récupère le code, `actions/setup-node@v4` installe Node. Le `@v4` est une version : on l'épingle, sinon la pipeline change de comportement sans qu'on ait rien touché.
- `needs: build` crée une **dépendance entre jobs**. Sans lui, GitHub lance tous les jobs en parallèle. Avec lui, `deploy` attend.
- Les **artefacts** transportent des fichiers d'un job à l'autre. Chaque job tourne sur une machine différente : ce que `build` a produit n'existe pas chez `deploy` tant qu'on ne l'a pas explicitement transmis. C'est la source d'étonnement numéro un quand on débute.

La durée. Une quarantaine de secondes en tout, dont l'essentiel dans `npm ci`. Le `cache: 'npm'` de `setup-node` fait la différence entre 30 secondes et 8 secondes sur cette étape.

Ce qui casse vraiment sur ce genre de pipeline. Presque jamais le build. En pratique, ce sont les permissions : le job `deploy` n'a pas le droit d'écrire sur GitHub Pages tant qu'on ne le lui a pas accordé explicitement dans les réglages du repo. Le message d'erreur parle de permissions, et on perd vingt minutes à relire son YAML alors que le problème n'y est pas.

Cas 2 : une API avec base de données, cinq stages, quatre minutes

Le contexte. Une petite API météo écrite en Python avec [FastAPI](#), qui stocke l'historique des relevés dans PostgreSQL. Elle a des tests unitaires, des tests d'intégration qui ont besoin d'une vraie base, et elle est livrée sous forme d'image Docker.

FastAPI ? Un framework web Python, l'équivalent d'Express côté Node. On n'écrit pas de Python dans ce cours : ce qui nous intéresse ici, c'est la forme de la pipeline, pas le langage.

```
name: API CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'
      - run: pip install ruff
      - run: ruff check .                # 12 secondes, et ça casse en premier

  test-unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'
      - run: pip install -r requirements.txt
      - run: pytest tests/unit -q

  test-integration:
    runs-on: ubuntu-latest
    # Un conteneur PostgreSQL démarré à côté du job, le temps du job.
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_PASSWORD: test
          POSTGRES_DB: weather_test
```

```

# Sans ce healthcheck, les tests démarrent avant la base et échouent.
options: >-
  --health-cmd pg_isready
  --health-interval 5s
  --health-retries 5

env:
  DATABASE_URL: postgresql://postgres:test@localhost:5432/weather_test

steps:
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
  with:
    python-version: '3.12'
    cache: 'pip'
- run: pip install -r requirements.txt
- run: pytest tests/integration -q

build-and-push:
  needs: [lint, test-unit, test-integration] # les trois doivent être verts
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
  - uses: actions/checkout@v4
  - uses: docker/login-action@v3
    with:
      username: ${ secrets.DOCKERHUB_USER }
      password: ${ secrets.DOCKERHUB_TOKEN }
  - uses: docker/build-push-action@v6
    with:
      push: true
      # Le sha du commit comme tag : on saura toujours quel code tourne.
      tags: monorg/weather-api:${ github.sha }

```

Ce qu'on lit dedans. Deux nouveautés majeures par rapport au cas 1 :

- **services:** démarre un conteneur à côté du job, pour la durée du job. Ici PostgreSQL. C'est exactement le `docker-compose.yml` de J1, mais géré par la CI et jetable. Le `--health-cmd pg_isready` est le même healthcheck qu'on a écrit en J1 : sans lui, les tests se lancent pendant que la base démarre encore, et échouent une fois sur trois. Cette panne intermittente est un classique.
- **Les secrets :** `${ secrets.DOCKERHUB_TOKEN }` lit une valeur enregistrée dans les réglages du repo, jamais dans le fichier. On y revient en détail plus tard dans le cours.

Notez aussi la structure de `needs: [lint, test-unit, test-integration]` : les trois jobs de vérification tournent **en parallèle** (ils ne dépendent de rien), et le build attend les trois. On ne paie donc que la durée du plus lent, pas la somme des trois.

`${ ... }` ? La syntaxe d'expression de GitHub Actions. Ce qui est entre les accolades est évalué au moment de l'exécution : un secret, une variable du contexte (`github.sha`, `github.ref`), le résultat d'une condition.

La durée. Environ quatre minutes au total, réparties ainsi :

Job	Durée	Ce qui pèse
lint	25 s	l'installation de l'outil, pas l'analyse
test-unit	1 min 10	l'installation des dépendances
test-integration	2 min 30	le démarrage de PostgreSQL, puis les tests
build-and-push	1 min 40	le build de l'image et son envoi

Les trois premiers tournant en parallèle, la pipeline complète tient en 2 min 30 + 1 min 40, soit un peu plus de quatre minutes. On est dans le budget de dix minutes vu plus tôt.

Ce qui casse vraiment. Le job `test-integration`, et presque toujours pour une raison de synchronisation ou d'adressage. Le piège d'adressage mérite d'être connu : dans un `services:` GitHub Actions, la base est joignable sur `localhost`, alors que dans un `docker-compose.yml` elle est joignable par son nom de service. Deux mondes proches, une règle différente, et une heure perdue si on ne le sait pas.

Cas 3 : un projet multi-services, un graphe de dépendances

Le contexte. Une plateforme de réservation découpée en trois morceaux dans le même dépôt : un frontend React, une passerelle API en Node, un service de facturation en Python. Chaque partie a ses tests, chacune produit sa propre image. C'est ce qu'on appelle un **monorepo**.

Monorepo ? Un seul dépôt Git qui contient plusieurs applications ou services, au lieu d'un dépôt par service. Pratique pour les changements qui traversent plusieurs services, plus exigeant côté CI : il faut éviter de tout reconstruire à chaque commit.

```
name: Platform

on: [push, pull_request]

jobs:
  # Première marche : ne relancer que ce qui a réellement changé.
  changes:
    runs-on: ubuntu-latest
    outputs:
      front: ${ steps.filter.outputs.front }
      gateway: ${ steps.filter.outputs.gateway }
      billing: ${ steps.filter.outputs.billing }
    steps:
      - uses: actions/checkout@v4
      - uses: dorny/paths-filter@v3
        id: filter
        with:
          filters: |
            front: ['apps/front/**']
            gateway: ['services/gateway/**']
            billing: ['services/billing/**']

  test-front:
    needs: changes
    if: needs.changes.outputs.front == 'true'
```

```

runs-on: ubuntu-latest
strategy:
  # Deux navigateurs, deux jobs lancés en parallèle.
  matrix:
    browser: [chromium, firefox]
steps:
  - uses: actions/checkout@v4
  - run: npx playwright test --project=${{ matrix.browser }}

test-gateway:
  needs: changes
  if: needs.changes.outputs.gateway == 'true'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: npm ci && npm test
      working-directory: services/gateway

test-billing:
  needs: changes
  if: needs.changes.outputs.billing == 'true'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: pip install -r requirements.txt && pytest
      working-directory: services/billing

# Le point de rendez-vous : un seul job à exiger sur la branche protégée.
ci-ok:
  needs: [test-front, test-gateway, test-billing]
  if: always()
  runs-on: ubuntu-latest
  steps:
    - run: |
        # "skipped" est acceptable, "failure" ne l'est pas.
        [[ "${{ contains(needs.*.result, 'failure') }}" == "false" ]]

```

Ce qu'on lit dedans. Trois idées de plus :

- **Le filtrage par chemin** : le premier job regarde quels fichiers ont changé et publie trois drapeaux. Les jobs suivants ne se déclenchent que si leur partie est concernée. Une correction de faute d'orthographe dans le frontend ne relance pas les tests de facturation. Sur un monorepo actif, c'est ce qui fait la différence entre une CI de 3 minutes et une CI de 25 minutes.
- **La matrice** : `strategy: matrix:` lance le même job plusieurs fois avec un paramètre différent. Ici deux navigateurs, en parallèle. On peut croiser plusieurs dimensions (versions de langage, systèmes d'exploitation), et le nombre de jobs se multiplie très vite.
- **Le job de rendez-vous** : quand des jobs sont conditionnels, on ne peut pas exiger "test-billing doit être vert" sur la branche protégée, puisqu'il ne tourne pas toujours. On ajoute donc un job final qui agrège les résultats, et c'est lui qu'on exige.

Matrice (matrix) ? Une façon de lancer le même job plusieurs fois avec des paramètres différents. Si l'app casse sur une version et marche sur l'autre, on le voit immédiatement, et sans écrire deux fois le job.

La durée. Très variable, et c'est le sujet : entre 90 secondes pour un commit qui ne touche qu'un service, et 6 minutes pour un commit qui touche tout. Sans le filtrage, ce serait 6 minutes à chaque fois.

Ce qui casse vraiment. Le `if:` avec les conditions, presque toujours. Un job conditionnel qui ne se déclenche jamais est pire qu'un job rouge : personne ne le remarque, et l'équipe croit être couverte par des tests qui ne tournent plus depuis six semaines. Le réflexe qui sauve : après avoir écrit une condition, provoquer volontairement le cas qui doit la déclencher, et vérifier dans l'interface que le job apparaît bien.

La progression, en une lecture

	Cas 1 : docs	Cas 2 : API	Cas 3 : plateforme
Jobs	2	4	6 et plus
Exécution	séquentielle	parallèle puis convergente	graphe conditionnel
Dépendances externes	aucune	PostgreSQL, DockerHub	plusieurs écosystèmes
Secrets	aucun	2	plusieurs
Durée	40 s	4 min	1 min 30 à 6 min
Le piège dominant	les permissions	la synchronisation avec la base	les conditions qui ne se déclenchent pas

La ligne à retenir : **la complexité d'une pipeline ne vient pas de la taille du code, elle vient du nombre de choses extérieures dont elle dépend.** Un projet de dix mille lignes sans dépendance a une pipeline plus simple qu'un projet de mille lignes qui parle à une base, à un registry et à un serveur.

On pousse plus loin : ce que la pipeline change dans la vie de l'équipe

On a vu au chapitre précédent ce qu'un bug coûte selon le moment où on l'attrape. Regardons maintenant l'autre bout de la lorgnette : ce qui change au quotidien, une fois la pipeline en place.

Sur les métriques DORA. Les quatre indicateurs bougent, mais pas au même rythme, et l'ordre est presque toujours le même :

- Le **lead time** chute immédiatement. Ce qui prenait une demi-journée de manipulations prend quatre minutes de machine. C'est le gain visible dès la première semaine.
- La **fréquence de déploiement** suit, mais avec un décalage. Elle ne monte que quand l'équipe *fait confiance* à la pipeline. Tant que quelqu'un vérifie encore à la main derrière le robot, on n'a rien gagné.
- Le **taux d'échec des changements** baisse ensuite, parce que les lots déployés deviennent petits. Un déploiement qui contient trois lignes casse rarement, et quand il casse, on sait quelles trois lignes.
- Le **temps de rétablissement** baisse en dernier, et pour une raison contre-intuitive : la meilleure façon de se rétablir vite, c'est de pouvoir redéployer la version précédente en un clic. Autrement dit, c'est la pipeline elle-même qui devient l'outil de secours.

Sur l'astreinte. L'astreinte, c'est le fait d'être joignable en dehors des heures de travail pour intervenir si le service tombe.

Astreinte (on-call) ? Un roulement dans l'équipe : une personne est d'astreinte pendant une semaine, avec un téléphone qui sonne si une alerte se déclenche, y compris à trois heures du matin.

Ce que la pipeline y change concrètement : sans elle, la personne d'astreinte doit connaître la procédure manuelle de déploiement, avoir les accès aux serveurs et enchaîner sept commandes sans se tromper, à trois heures du matin, sous stress. Avec elle, elle relance un déploiement de la version précédente et retourne se coucher. C'est le genre de détail qui ne figure sur aucune fiche de poste et qui décide si les gens restent dans une équipe.

Sur les gens qui arrivent. Une nouvelle personne dans l'équipe fait son premier déploiement le premier jour, en toute sécurité : la pipeline refusera de livrer si quelque chose ne va pas. Sans pipeline, le premier déploiement se fait sous surveillance, après plusieurs semaines, et reste un moment redouté.

Le point d'honnêteté, parce qu'il compte. Une pipeline coûte aussi. Elle demande de l'entretien : des dépendances d'actions à mettre à jour, des tests instables à réparer, du temps de calcul à payer. Et surtout, elle ne remplace aucune décision : elle exécute ce qu'on lui a écrit. Une pipeline qui déploie du code non testé le fait très efficacement, à grande échelle, plusieurs fois par jour.

Le vocabulaire est posé, les mécanismes sont vus, les pièges sont nommés. Il reste une dimension dont on n'a pas parlé et qui, elle, ne se rattrape jamais après coup : la sécurité.

8 - DevSecOps

L'écart entre les deux réponses est le sujet de ce chapitre. Un projet Node de deux mille lignes embarque couramment plusieurs centaines de paquets, écrits par des gens qu'on ne connaît pas, mis à jour sans qu'on le sache, et qui s'exécutent avec exactement les mêmes droits que notre propre code.

Le **DevSecOps**, c'est DevOps avec le **Sec** (sécurité) injecté *dans* la pipeline, au lieu d'être un contrôle qu'on fait à la fin (trop tard).

DevSecOps ? "Shift left security" : on déplace les contrôles de sécurité vers la gauche, c'est-à-dire tôt dans le cycle, directement dans la pipeline. Référence : [Red Hat - What is DevSecOps](#).

Concrètement, on ajoute des jobs de sécurité à la pipeline :

- **SAST** (Static Application Security Testing) : un scanner lit votre code source et repère les patterns dangereux (injection SQL, secrets en dur).
- **Dependency scanning** : on vérifie que vos dépendances npm n'ont pas de failles connues (`npm audit` fait déjà une partie du job).
- **Container scanning** : on scanne l'image Docker pour détecter des CVE dans les paquets système.

CVE ? Common Vulnerabilities and Exposures : un identifiant public pour une faille de sécurité connue. Exemple : CVE-2021-44228 (la fameuse Log4Shell).

L'essentiel du shift left

Pourquoi "vers la gauche" change tout

Le modèle classique, celui qui a dominé pendant vingt ans : l'équipe développe pendant six mois, puis un audit de sécurité a lieu avant la mise en production. Un rapport arrive, il liste quarante-sept problèmes, et la date de livraison est dans deux semaines.

Ce qui se passe alors est parfaitement prévisible : on corrige les cinq problèmes les plus graves, on négocie les autres, et on livre. Les quarante-deux restants deviennent de la dette, souvent définitive. Non pas parce que l'équipe s'en moque, mais parce que corriger un choix d'architecture six mois après l'avoir fait coûte cent fois plus cher que de le corriger le jour même.

Le shift left ne change pas la nature des contrôles. Il change **le moment**, et donc le coût de la correction. On retrouve exactement la logique du tableau vu au chapitre 5 : le même défaut, à des prix différents selon où on le rencontre. Un scanner qui bloque une pull request coûte cinq minutes. Le même problème découvert en audit final coûte une renégociation de planning.

Un mot sur l'objection légitime : *si on bloque à chaque alerte, on ne livre plus rien*. C'est vrai, et c'est la principale raison pour laquelle les équipes finissent par désactiver leurs scanners. La réponse tient en un réglage : on bloque sur les niveaux `HIGH` et `CRITICAL`, on signale le reste sans bloquer. Un scanner qui crie tout le temps est un scanner qu'on ignore.

La famille des scanners, un par question

Chaque type de scanner répond à une question différente. Les confondre, c'est croire qu'on est couvert alors qu'on ne l'est pas.

Type	La question posée	Un outil pour commencer
SAST	mon propre code contient-il un motif dangereux ?	Semgrep
SCA	mes dépendances ont-elles des failles connues ?	npm audit , Dependabot
Secret scanning	ai-je committé une clé par accident ?	gitleaks
Container scanning	l'image que je publie contient-elle des paquets système vulnérables ?	Trivy
DAST	l'application qui tourne se laisse-t-elle attaquer ?	OWASP ZAP
SBOM	qu'est-ce qu'il y a exactement dans ce que je livre ?	Syft

Quelques définitions, parce que les acronymes tombent vite :

SAST ? Static Application Security Testing. On analyse le code **sans l'exécuter**, à la recherche de motifs connus pour être dangereux : une requête SQL construite par concaténation, un mot de passe écrit en dur, une désérialisation non contrôlée.

SCA ? Software Composition Analysis. On ne regarde pas le code, on regarde la **liste des dépendances**, et on la compare aux bases de vulnérabilités publiques. C'est ce que fait `npm audit`, et c'est le scanner qui trouve le plus de choses en pratique, tout simplement parce que la majorité du code livré n'a pas été écrite par l'équipe.

DAST ? Dynamic Application Security Testing. Cette fois l'application **tourne vraiment**, et le scanner l'attaque : il envoie des requêtes malformées, tente des injections, cherche des pages accessibles sans authentification. Complémentaire du SAST, parce qu'il voit ce que le code ne dit pas.

SBOM ? Software Bill of Materials, la nomenclature du logiciel. C'est la liste complète et lisible par une machine de tout ce que contient un livrable : chaque bibliothèque, chaque version, chaque licence. Formats standards : [CycloneDX](#) et [SPDX](#).

Le SBOM mérite un mot de plus, parce que c'est le plus récent et le moins intuitif. Son intérêt n'apparaît pas le jour où on le génère, mais le jour où une faille critique est annoncée dans une bibliothèque très répandue. La question devient alors : *est-ce qu'on l'utilise, et où ?* Sans SBOM, la réponse demande plusieurs jours de recherche dans des dizaines de dépôts. Avec, c'est une recherche dans un fichier. Lors de la faille Log4Shell en décembre 2021, cet écart s'est mesuré en jours d'exposition, et c'est ce qui a poussé plusieurs législations à rendre le SBOM obligatoire pour les fournisseurs de logiciel.

On a déjà croisé Trivy en J1, quand on a scanné notre image. La vraie nouveauté du DevSecOps, c'est de ne plus le lancer à la main mais d'en faire un job qui tourne à chaque push, avec un seuil qui bloque.

Trois attaques qui ont changé les pratiques

Ces trois histoires expliquent, mieux que n'importe quel argumentaire, pourquoi le sujet est devenu central. Elles ont un point commun : dans les trois cas, le code des victimes était impeccable.

event-stream, novembre 2018. `event-stream` est un paquet npm utilitaire, téléchargé plus d'un million de fois par semaine, maintenu bénévolement par une personne qui n'y touchait plus depuis longtemps. Quelqu'un se propose gentiment pour reprendre la maintenance. Le mainteneur accepte, ravi : c'est le fonctionnement normal de l'open source. Quelques semaines plus tard, le nouveau responsable ajoute une dépendance d'apparence anodine, `flatmap-stream`, qui contient du code chiffré. Ce code ne s'active que dans un contexte précis : celui du portefeuille de cryptomonnaie Copay, où il extrait les clés privées des comptes dépassant un certain solde. La charge est restée en place deux mois et demi avant qu'un utilisateur ne remarque la dépendance étrange. Le fil de discussion sur le dépôt est [toujours en ligne](#), et il vaut la lecture : c'est un des documents les plus honnêtes qui existent sur la fragilité du modèle. La leçon retenue par l'industrie : **le maillon faible n'est pas le code, c'est le transfert de confiance entre humains.**

SolarWinds, décembre 2020. SolarWinds édite Orion, un logiciel de supervision réseau utilisé par des grandes entreprises et des administrations américaines. Des attaquants pénètrent l'infrastructure de compilation de l'éditeur et modifient le code **au moment du build**, pas dans le dépôt source. Le logiciel produit est donc signé numériquement par SolarWinds, parfaitement authentique du point de vue des clients, et porteur d'une porte dérobée. Il est distribué comme mise à jour officielle à environ dix-huit mille organisations. La leçon : **la chaîne de build est elle-même une cible, et une signature ne prouve que l'origine, pas l'innocuité.** C'est de cet épisode que viennent les travaux actuels sur la provenance des artefacts, comme le cadre [SLSA](#).

xz-utils, mars 2024. Celle-ci est la plus vertigineuse. `xz-utils` est une bibliothèque de compression présente dans à peu près toutes les distributions Linux. Pendant près de trois ans, une personne contribue au projet, avec des contributions de bonne qualité, gagne progressivement la confiance du mainteneur unique et épuisé, et obtient les droits de publication. Elle glisse alors une porte dérobée dans les archives de publication (pas dans le dépôt Git, ce qui la rendait invisible à la simple lecture du code), permettant à qui possédait une clé

précise de se connecter en SSH sur n'importe quelle machine affectée. La faille reçoit l'identifiant CVE-2024-3094 et le score de gravité maximal. Elle a été découverte **par accident** : [Andres Freund](#), un développeur PostgreSQL, trouvait que ses connexions SSH prenaient 500 millisecondes au lieu de 100, et a voulu savoir pourquoi. Les versions piégées n'avaient pas encore atteint les distributions stables. La leçon, et elle est inconfortable : **une infrastructure mondiale reposait sur un mainteneur bénévole seul, et sa fatigue était le vecteur d'attaque.**

Ce qu'on retient des trois : aucune n'aurait été empêchée par un développeur plus prudent dans son propre code. C'est bien pour ça que la sécurité devient une propriété de la chaîne de livraison, et donc un sujet de pipeline.

À quoi ça ressemble dans un fichier

Un job de sécurité n'est pas plus compliqué qu'un job de test. Voilà les deux premiers à mettre dans n'importe quel projet :

```
security:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    # 1. Les dépendances : on bloque sur "high" et au-dessus,
    #   sinon la moindre alerte mineure arrête toute l'équipe.
    - run: npm audit --audit-level=high

    # 2. Les secrets : gitleaks relit tout l'historique du dépôt,
    #   pas seulement le dernier commit.
    - uses: gitleaks/gitleaks-action@v2
```

Un détail du deuxième point mérite d'être souligné, parce qu'il surprend toujours : gitleaks relit **l'historique**. Un secret committé puis supprimé au commit suivant est toujours dans le dépôt, récupérable par n'importe qui l'a cloné. Un `git rm` ne le fait pas disparaître.

Ce qui mène à la seule bonne réaction quand une clé fuite, et elle n'est pas dans Git : **la clé est morte, on la révoque et on en génère une nouvelle**. Nettoyer l'historique est une opération secondaire, jamais la première.

On ne fera pas de DevSecOps complet aujourd'hui, mais gardez l'idée : un `npm audit` dans votre pipeline, c'est déjà du DevSecOps. Vous y toucherez du doigt plus tard dans le cours, quand on parlera des secrets et du déploiement.

On va plus loin : la sécurité comme culture, pas comme portail

La transformation la plus profonde n'est pas dans les outils, elle est dans la répartition des responsabilités.

Le modèle du portail : une équipe sécurité, séparée, qui autorise ou refuse les mises en production. Elle est perçue comme un frein, elle intervient trop tard pour changer quoi que ce soit d'architectural, et les équipes de développement apprennent à la contourner ("on fera l'audit après la livraison"). Résultat : la sécurité est vécue comme une contrainte administrative.

Le modèle du garde-fou : la sécurité fournit des outils, des règles par défaut et de l'expertise, et ce sont les équipes qui portent la responsabilité de leur service. Le scanner tourne dans leur pipeline, l'alerte arrive chez elles, et personne n'attend l'autorisation de quelqu'un d'autre.

Ce changement porte un nom dans les organisations qui l'ont formalisé : les **security champions**, des développeurs volontaires dans chaque équipe, formés un peu plus que les autres, qui font le lien avec l'équipe sécurité. Ce n'est pas un poste, c'est un rôle en plus. Et c'est souvent une excellente porte d'entrée pour un profil junior qui veut se différencier : c'est une responsabilité qu'on peut prendre tôt, et qui se remarque.

Deux réflexes utiles pour la suite du parcours, plus utiles que n'importe quel outil :

- **Le principe du moindre privilège** : chaque composant reçoit exactement les droits dont il a besoin, et rien de plus. On l'a appliqué en J1 sans le nommer, avec l'utilisateur non-root dans le Dockerfile. Un jeton de CI qui ne fait que pousser des images n'a aucune raison de pouvoir supprimer le dépôt.
- **Le post-mortem sans blâme** : après un incident, on écrit ce qui s'est passé, en cherchant la cause systémique et pas le coupable. Une organisation qui cherche des coupables obtient des gens qui cachent les incidents, ce qui est bien pire que l'incident.

Notre chaîne sait maintenant construire, tester, vérifier et livrer une image. Il reste une question, et c'est la dernière de la journée : une fois l'image prête, qui la fait tourner, et qui la relance à trois heures du matin quand elle meurt ?

9 - Vers Kubernetes : introduction à K3S

Aujourd'hui, notre pipeline sait construire et tester une image Docker automatiquement. Il reste une question : où est-ce qu'on la fait *tourner*, une fois qu'elle est prête ? Un `docker run` sur un seul serveur, ça marche pour un projet de cours. Mais dès qu'il faut plusieurs instances de l'app, de la résilience (un conteneur qui plante redémarre tout seul), et des mises à jour sans coupure de service, un simple `docker run` ne suffit plus. C'est le rôle d'un **orchestrateur de conteneurs**, et le plus utilisé au monde s'appelle **Kubernetes** (souvent abrégé **K8s**).

L'essentiel

Kubernetes ? Un système qui décide où et comment faire tourner vos conteneurs sur un ensemble de machines, les relance s'ils plantent, les met à jour sans interruption, et les fait grossir ou rétrécir selon la charge. Vous décrivez l'état voulu ("je veux 3 instances de mon API qui tournent"), Kubernetes se charge de le maintenir.

Le souci, c'est que Kubernetes "complet" est une bête assez lourde à installer et à maintenir : plusieurs composants (etcd, API server, scheduler, controller manager...), pensé pour des clusters de production à grande échelle. Pas idéal pour apprendre ou pour un petit projet.

C'est là qu'intervient **K3S** : une distribution **allégée** de Kubernetes, créée par Rancher, qui tient dans un seul binaire de moins de 100 Mo. K3S retire ce qui n'est pas essentiel (drivers cloud spécifiques, stockage etcd remplacé par SQLite par défaut) tout en restant un vrai Kubernetes conforme aux standards : ce que vous apprenez avec K3S s'applique tel quel à un cluster Kubernetes "classique" en entreprise.

K3S ? Une version légère de Kubernetes (le nom joue sur "K8s moins la moitié des lettres" : K-3-S). Un seul binaire à installer, parfait pour apprendre, pour un petit serveur, ou pour de l'embarqué/IoT. Doc : k3s.io.

Pour l'instant, reprenez juste l'idée : c'est l'outil qui prendra le relais de la pipeline qu'on construit aujourd'hui, pour faire tourner la Todo API en continu, la scaler, et la mettre à jour sans interruption. On installera K3S et on y déploiera vraiment la Todo API plus tard dans le cours.

Pour les curieux : Kubernetes a-t-il gagné ?

Kubernetes n'a pas toujours été évident, et il n'a pas toujours été seul. Trois concurrents sérieux ont existé, et savoir pourquoi ils ont perdu aide à comprendre ce que Kubernetes apporte vraiment.

- [Docker Swarm](#) : l'orchestrateur intégré à Docker. Sa syntaxe ressemble beaucoup à celle de `docker-compose`, ce qui le rend très facile à prendre en main. Il fonctionne toujours, il est toujours maintenu, et il reste un choix raisonnable pour une petite infrastructure. Il a perdu la bataille de l'écosystème : quand un fournisseur cloud, un éditeur d'outil de monitoring ou un projet open source ajoute un support d'orchestrateur, il le fait pour Kubernetes.
- [Apache Mesos](#) : plus ancien, plus généraliste, capable d'orchestrer autre chose que des conteneurs. Il a fait tourner Twitter et Airbnb à une époque. Trop complexe pour le cas courant, il a été largement abandonné.
- [HashiCorp Nomad](#) : nettement plus simple que Kubernetes, et capable d'orchestrer aussi bien des conteneurs que des binaires ou des machines virtuelles. Il conserve une base d'utilisateurs convaincus, mais reste marginal.

La raison de fond de la victoire de Kubernetes n'est pas technique, elle est stratégique. Google l'a donné à une fondation neutre, la [CNCF](#), plutôt que de le garder. Résultat : aucun fournisseur cloud n'était en train d'adopter l'outil d'un concurrent, et tous ont pu proposer leur Kubernetes managé. C'est ce qui a fait de Kubernetes le langage commun de l'industrie, avec un effet secondaire très concret pour une carrière : les compétences Kubernetes sont transférables d'un employeur à l'autre, ce qui est loin d'être le cas de tous les outils d'infrastructure.

Un mot d'honnêteté pour finir, parce qu'on va y passer une journée entière : Kubernetes est un outil puissant et coûteux en complexité. Pour une application unique sur un seul serveur, un `docker compose up` bien fait reste la bonne réponse. On y va parce que c'est le standard du métier et parce que c'est demandé partout, pas parce que c'est toujours la solution la plus adaptée.

10 - À votre tour : ta première pipeline verte (après-midi)

Cet après-midi, on fait passer une **première pipeline au vert**. Pas besoin de prendre la Todo API (trop de pièces mobiles : base de données, secrets...). On va rester simple : **ClickFast**, un petit jeu de clics en HTML/CSS/JS, servi par nginx. Le but, c'est juste de voir le voyant vert s'allumer pour la première fois. 😎

Le warm-up ClickFast : un site statique (HTML/CSS/JS), zéro base de données, zéro secret. La pipeline la plus minimale possible. Une fois qu'on a vu passer celle-là au vert, vous attaquerez la vraie Todo API avec confiance, plus tard dans le cours.

On utilise **GitHub Actions** aujourd'hui : c'est directement intégré à votre repo GitHub, rien à installer, et la syntaxe est parmi les plus accessibles pour un premier contact avec la CI. Plus tard sur la vraie Todo API ce sera le même principe.

Le projet

À la fin de l'après-midi, le repo contient :

- **ClickFast dockerisé** : un `Dockerfile` qui produit une image servant le jeu, et qui tourne en local sur un port de votre choix.
- **Des tests qui passent** : les tests du compteur de clics, lancés par `npm test`, verts en local comme en CI.
- **Un ou plusieurs workflows** dans `.github/workflows/`, qui se déclenchent à chaque push, lancent les tests puis construisent l'image, et refusent d'aller plus loin si les tests cassent.
- **Un badge de statut** en haut du README, qui affiche en vert ou en rouge l'état de la dernière exécution.
- **Un README** avec le nom, ce que fait le projet, comment le lancer.

L'objectif final tient en une phrase : *un `git push`, et une machine que vous ne verrez jamais teste et construit votre projet, puis vous dit oui ou non.*

Pas besoin de suivre les 8 phases ci-dessous, [suivez le projet ici](#).

Seulement après avoir fini les exercices essentiels, je vous file les exercices bonus (la partie API de ClickFast peut être sautée).

Récapitulatif Jour 2

Ce qu'on a appris aujourd'hui

1. **Diagnostiquer avant d'automatiser** : lire un message d'erreur Docker, distinguer `EXPOSE` (de la documentation) de `ports` (une exposition réelle), et savoir quelle commande répond à quelle question.
2. **DevOps, CI et CD** : le DevOps est une culture (collaboration Dev/Ops), la CI teste automatiquement chaque changement, la CD automatise la livraison jusqu'à la prod (avec ou sans validation humaine). CALMS en donne la version détaillée, et vingt ans d'histoire expliquent pourquoi le mouvement est né.
3. **Les principes de l'intégration continue** : une branche de référence toujours déployable, des intégrations quotidiennes, un artefact construit une fois et promu, un budget de dix minutes, et des feature flags pour découpler le déploiement de la mise à disposition.
4. **Anatomie d'une pipeline** : une suite de stages (lint, test, build, push, deploy) qui s'arrête au premier échec, le fail fast, articulée avec les environnements (dev/staging/prod) et les branches Git.
5. **Trois pipelines réelles** : d'un site statique en 40 secondes à un monorepo avec graphe conditionnel, et le constat que la complexité vient des dépendances extérieures, pas de la taille du code.
6. **DevSecOps** : le shift left, la famille des scanners (SAST, SCA, DAST, secrets, conteneurs, SBOM), et trois attaques de chaîne d'approvisionnement où le code des victimes était pourtant impeccable.
7. **Première pipeline GitHub Actions** : ClickFast dockerisé, testé, buildé, avec une branche protégée et une durée mesurée.

Points d'attention pour demain

- **Finissez votre pipeline ClickFast** si elle n'est pas encore complètement verte : c'est la base pour la suite.
- **Gardez votre Journal de bord ouvert** : on continue de le remplir demain, il fait partie du rendu de fin de semaine.
- **Relisez le tableau de la phase 8** : demain, les mêmes questions de durée se poseront sur une pipeline nettement plus lourde.
- **Demain on attaque GitLab-CI sur la vraie Todo API** : tests avec base de données, cache, secrets, build et push DockerHub, plus Prometheus et Grafana pour commencer à mesurer ce qui tourne !